AI 协作反思日志 - Phase 3 详细设计

团队： 不是bug是特性
日期： 2026.5.5

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
任务1：类图设计	Gemini 3	基于 CampusHub 的需求文档与架构设计，生成核心模块的完整类图。	3	40%
任务2：API设计	Gemini 3	根据 CampusHub 业务流程生成核心接口的 RESTful API 规范文档（OpenAPI 格式）。	3	50%
任务3：数据库设计	Gemini 3 Flash	生成核心实体的 ER 图结构及 MySQL 建表 SQL，包含索引设计。	4	20%

2. AI 助力分析 (Q1)

- **框架与骨架的高效构建**：AI 表现出极强的"冷启动"能力，能根据底层的业务需求快速搭建出类图骨架、API 接口定义以及标准的建表 SQL 语句，直接帮团队节省了约 60% 的机械性编写工作 。
- **技术决策的表格化对比**：在设计模式的选择与数据库范式审查中，通过要求 AI 进行"维度化输出"，AI 以清晰的表格形式对比了不同设计模式（如状态模式与策略模式）的优劣势，使团队能以极高的效率完成架构微调。
- **边界异常的初步排查**：在数据库审查中，AI 能够快速指出潜在的性能瓶颈（如高频查询字段缺少索引），帮助团队在编码前规避了部分底层设计缺陷。

3. AI 误导分析 (Q2)

- **SOLID 检查中发现的 AI 设计问题数量：7 处**
- **API 审查中发现的 AI 设计问题数量：16 处**
- **最严重的设计问题是：** AI 存在严重的 **"快乐路径（Happy Path）"依赖与安全/职责盲区** 。在类图设计中，AI 极其容易将多个子域逻辑耦合进单个 Service（如 `OrderService` 严重违反 SRP），或错误地使用继承破坏父类契约（如 `UrgentDemand` 违反 LSP） 。在 API 设计中，AI 系统性地遗漏了并发接单冲突（40901）等异常处理 ，且完全缺乏对敏感数据（如手机号、邮箱）的脱敏与鉴权安全意识 。

4. Prompt 改进计划 (Q3)

- **上阶段改进措施的验证结果：** 上阶段提出的 **"维度化输出"** 策略在本阶段得到了完美的验证。要求 AI 必须以表格形式进行优劣对比，成功避免了大量冗长的文字叙述，帮助团队在 API 规范审查和数据库索引设计上大幅提升了决策效率。

- **本阶段新的改进措施：** 针对 AI 在本阶段暴露出的“倾向于宽松定义”以及“系统性遗漏负向/异常路径”的缺陷，团队决定正式在后续的开发与测试 Prompt 中**引入“负向约束”**。在下发任务时，不仅告诉 AI “要做什么”，更要明确制定反向边界（例如：“*必须包含至少3种业务冲突的错误响应*”、“*禁止在响应示例中明文输出隐私字段*”、“*不使用分布式事务*”）。

---

5. 本阶段的核心工程判断

- **决策内容：**
  1. **放弃继承，采用组合重构加急需求逻辑：**将 `UrgentDemand` 子类彻底移除，改为在 `Demand` 实体中内嵌 `isUrgent` 和 `urgentDeadline` 字段，并在 Service 层通过自定义 `Comparator` 统一处理排序。
  2. **确立跨模块的 Service 隔离规范：**严禁高层业务直接依赖低层具体实现，规定跨模块访问必须走 Service 接口，绝对禁止 `OrderService` 直接注入其他模块的 `DemandRepository`。
- **为什么 AI 无法做这个决策：** AI 缺乏长期的项目维护全局观，它在生成代码时倾向于选择“当时最快、最直观”的实现方式（如直接 `extends` 或直接 `new` 实例），从而导致隐蔽的耦合。AI 无法感知项目在未来引入新状态（如订单“争议中”）或新板块（如“表白墙”）时的扩展成本，这种对工程规范、高内聚低耦合的“代码洁癖”与长期演进的权衡，必须依赖人工的工程经验进行注入与把关。